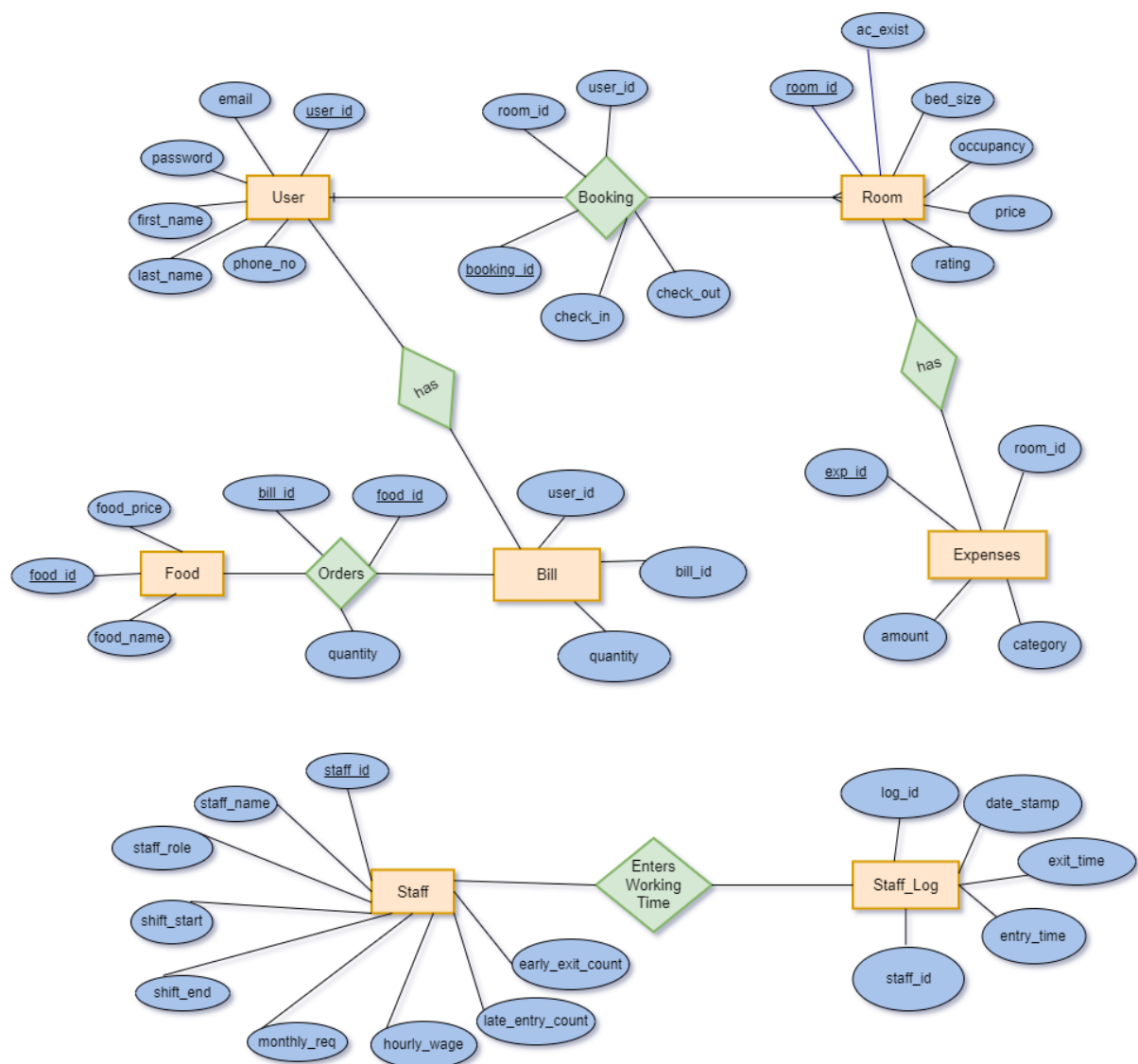


DBMS Mini-Project

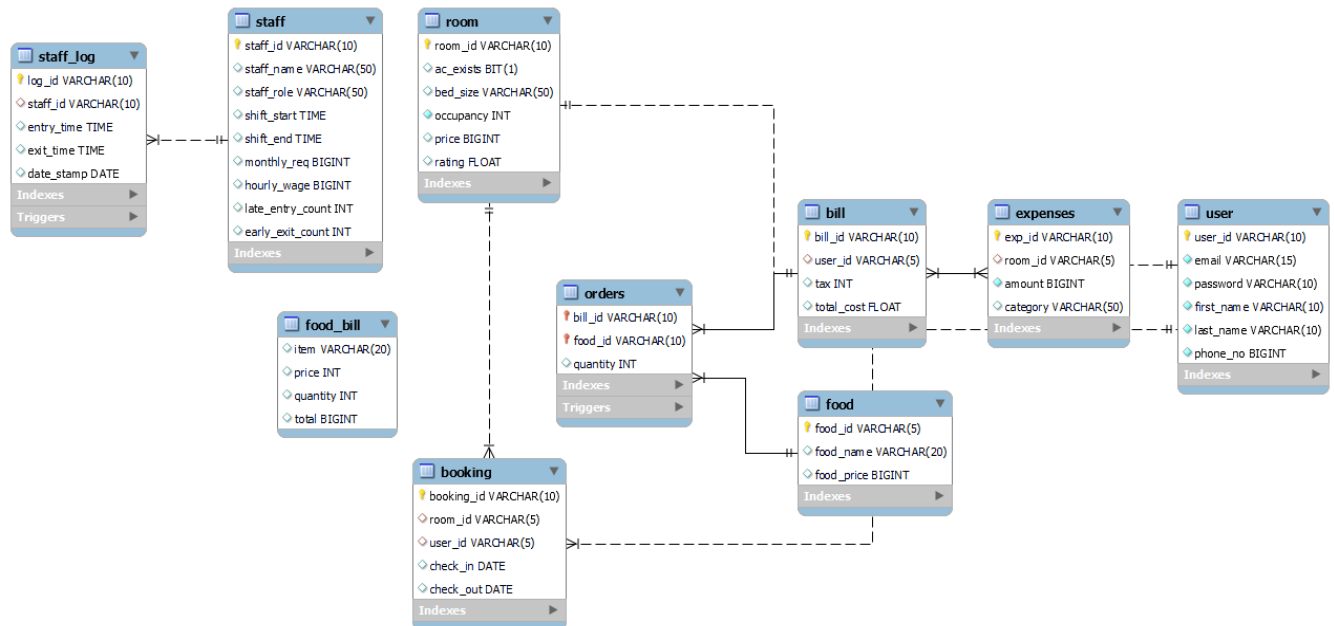
By Rishikesh Devanathan (2001CS85), Sai Vardhan Vemulapalli (2001CS58), Angelina Shibu (2001CS06), NV Vineeth (2001CS49)

Guest House Booking

ER Diagram



Relational Diagram



Procedures, Triggers, Functions and Indices

Room Booking - Procedure

We can obtain the rooms available for booking by left joining the Booking table with the Room table on the room_id attribute. We then filter the joined table on the user's specification which is inputted into the procedure as ac_exists, bed_size, and occupancy. Next, we make sure that the dates are possible to be booked by checking either

- The room hasn't been booked in the past before: If the room is being booked for the first time, it will not contain an entry in the Booking table which means that the booking_id for it will be NULL.
- The room is available for the specified check_in and check_out date: If the room is available for the given user date range then the following condition satisfies

```
BOOKING.check_in <= check_in and BOOKING.check_out >= check_out
```

For each room in the Booking table. We must ensure that we don't provide the table which will be booked in the future during the given specified range. This is done through the subquery:

```
ROOM.room_id not in
```

```
(select ROOM.room_id from ROOM LEFT JOIN BOOKING on ROOM.room_id
= BOOKING.room_id
where (BOOKING.check_in <= check_in and BOOKING.check_out >=
check_out))
```

```
create procedure get_available_rooms(
  IN ac_exists bit(1),
  IN bed_size varchar(50),
  IN occupancy int,
  IN check_in date,
  IN check_out date
)
begin
  select distinct(ROOM.room_id)
  from ROOM LEFT JOIN BOOKING on ROOM.room_id = BOOKING.room_id
  where ((ROOM.ac_exists = ac_exists and ROOM.bed_size = bed_size and
ROOM.occupancy = occupancy)
and ((BOOKING.booking_id IS NULL)
or (ROOM.room_id not in
(select ROOM.room_id from ROOM LEFT JOIN BOOKING on ROOM.room_id =
BOOKING.room_id
where (BOOKING.check_in <= check_in and BOOKING.check_out >=
check_out)))));
end |
```

Staff Salary - Function

```
create function calc_working_hours(staff_id varchar(5)) returns time
deterministic
begin
  declare total_worked time;
  set total_worked = (select time(sum(TIMEDIFF(STAFF_LOG.exit_time,
STAFF_LOG.entry_time)))
from STAFF_LOG
where STAFF_LOG.staff_id = staff_id);
  return total_worked;
end;|
```

We create a function to determine the amount of time in hours a staff has worked according to the entry and exit times in the STAFF_LOG table.

```
create function calc_salary(staff_id varchar(5)) returns bigint
deterministic
begin
```

```

declare worked_hours bigint;
declare wage int;
declare salary bigint;
set worked_hours = HOUR(calc_working_hours(staff_id));
set wage = (select STAFF.hourly_wage from STAFF where STAFF.staff_id =
staff_id);
set salary = worked_hours*wage;
return salary;
end;|

```

Using the hours worked which is calculated by invoking the previous function, we can find the salary to be given to the staff by multiplying it with the hourly wage which is fixed for the staff. I.e.,

$$\text{Salary} = (\text{Hours worked according to Log}) \times (\text{Hourly wage})$$

Bill Generation/View - Function, Procedure, Trigger

The bill generation for a user is done in the following steps:

1. Loop through the Booking table, and for every booking of the user, calculate the bill for the room by invoking the **calc_room_bill()** function
2. Loop through the Bill table, and for every bill of the user, calculate the grand total for the food contained in that bill by invoking the **calc_food_bill()** function
3. Take the sum of steps 1 and 2

```

-- sums all the food and room bills of an user
create function calc_total_bill(u_id varchar(10)) returns float deterministic
begin
declare cnt int default 0;
declare sum_total float default 0;
declare total int default (select count(*) from BILL where BILL.user_id = u_id);
declare b_id varchar(5);
declare cur cursor for select BILL.bill_id from BILL where BILL.user_id = u_id;
declare cur2 cursor for select BOOKING.booking_id from BOOKING where BOOKING.user_id = u_id;

open cur;
while cnt < total do
fetch cur into b_id;
set sum_total = sum_total + calc_food_bill(b_id, 10);
set cnt = cnt + 1;
end while;
close cur;

set total = (select count(*) from BOOKING where BOOKING.user_id = u_id);
set cnt = 0;

open cur2;
while cnt < total do
fetch cur2 into b_id;
set sum_total = sum_total + calc_room_bill(b_id, 10);
set cnt = cnt + 1;
end while;
close cur2;

return sum_total;
end|

```

Note that we can determine the tax for the room and food separately.

calc_room_bill() Function

We can calculate the room bill as:

$$\text{Room bill} = (\text{price per night}) \times (\text{number_of_days}(\text{check_out} - \text{check_in}))$$

The number of days is calculated using the datediff function in mysql and room price is fetched from the Room table.

```

create function calc_room_bill(b_id varchar(10), tax int) returns bigint
deterministic

```

```

begin
  declare sum_total bigint;
  declare rprice bigint;
  declare num_days bigint;
  set num_days = (select datediff(BOOKING.check_out, BOOKING.check_in)
from BOOKING where BOOKING.booking_id = b_id);
  set rprice = (select ROOM.price from BOOKING natural join ROOM where
BOOKING.booking_id = b_id);
  set sum_total = rprice*num_days;
  set sum_total = sum_total + sum_total*(tax/100);
  return sum_total;
end |

```

calc_food_bill() Function

The food bill is calculated by looping through the ORDERS table with the same bill id and calculating the price of the food by natural joining the FOOD table with ORDERS table to obtain the price. The quantity ordered is obtained simply from the ORDERS table.

```

create function calc_food_bill(b_id varchar(10), tax int) returns float
deterministic
begin
  declare cnt int default 0;
  declare sum_total float default 0;
  declare total int default (select count(*) from ORDERS where
ORDERS.bill_id = b_id);
  declare price float;
  declare quant int;
  declare cur cursor for select FOOD.food_price, ORDERS.quantity from
ORDERS natural join FOOD where ORDERS.bill_id = b_id;

  open cur;
  while cnt < total do
    fetch cur into price, quant;
    set sum_total = sum_total + price*quant;
    set cnt = cnt + 1;
  end while;
  close cur;

  set sum_total = sum_total + sum_total*(tax/100);

  return sum_total;
end |

```

Bill Generation View - Procedure

The bill is generated for a given user using the two procedures `show_food_bill` and `show_room_bill`. This is done by invoking the `calc_food_bill` and `calc_room_bill` functions to obtain the final grand total. Furthermore, the food item list and room information list is also given with the user information.

`show_food_bill()` Procedure, Trigger

We create a trigger to add an empty bill to the BILL table once we create orders that map a bill to a food item. This is because the order can't exist without the foreign key existing in the database already.

```
create trigger insert_bill before insert on ORDERS
for each row
begin
    if (not (exists(select * from BILL where BILL.bill_id =
NEW.bill_id))) then
        insert into BILL (bill_id) values (NEW.bill_id);
    end if;
end;|
```

We format the food bill by creating a new FOOD_BILL inside the procedure and adding all the food item information to this table as we scan the ORDERS table.

We then print this table along with the taxed grand total and the information about the user who ordered it.

```
create procedure show_food_bill(
    IN b_id varchar(10),
    IN u_id varchar(5),
    IN tax int)
begin
    declare cnt int default 0;
    declare total int default (select count(*) from ORDERS where
ORDERS.bill_id = b_id);
    declare fname varchar(100);
    declare price float;
    declare quant int;
    declare cur cursor for select FOOD.food_name, FOOD.food_price,
ORDERS.quantity from ORDERS natural join FOOD where ORDERS.bill_id =
b_id;
    open cur;

    create table if not exists FOOD_BILL (
        item varchar(20),
        price int,
```

```

    quantity int,
    total bigint
);

-- clear the entry of the bill
delete from FOOD_BILL;

while cnt < total do
    fetch cur into fname, price, quant;
    insert into FOOD_BILL values (fname, price, quant, price*quant);
    set cnt = cnt + 1;
end while;
close cur;

update BILL
set BILL.total_cost = calc_food_bill(b_id, tax),
BILL.user_id = u_id,
BILL.tax = tax
where BILL.bill_id = b_id;

select user_id, first_name, last_name, phone_no from USER where
USER.user_id = u_id;
select * from FOOD_BILL;
select total_cost as "Grand Total" from BILL where BILL.bill_id =
b_id;
end |

```

Food Bill View Example

```
mysql> call show_food_bill('b101', 'u103', 10);
```

user_id	first_name	last_name	phone_no
u103	James	James	2074166419

```
1 row in set (0.04 sec)
```


item	price	quantity	total
Thukpa	160	2	320
Bamboo shoot pork	170	4	680
Galouti kebab	150	5	750
Jadoh	150	5	750
Butter chicken	120	1	120
Chicken pakora	210	1	210
Dal baati churma	170	3	510
Appam and chicken st	200	5	1000
Masala coke/pepsi	90	1	90
Sorcshe ilish	110	3	330
Haleem	190	2	380

```
11 rows in set (0.04 sec)
```


Grand Total
5654

```
1 row in set (0.05 sec)
```



```
Query OK, 0 rows affected (0.05 sec)
```

show_room_bill() Procedure

We show the room bill by showing the booking entry along with the user information and the grand total of the room booking by calling the calc_room_bill() function.

```
create procedure show_room_bill(
  IN b_id varchar(10),
  IN tax int)
begin
  declare u_id varchar(50);
  set u_id = (select user_id from BOOKING where BOOKING.booking_id =
b_id);
  select user_id, first_name, last_name, phone_no from USER where
USER.user_id = u_id;
  select * from BOOKING where BOOKING.booking_id = b_id;
  select tax as "Tax Percent";
  select calc_room_bill(b_id, tax) as "Grand Total" from BILL where
BILL.bill_id = b_id;
```


end |

Room Bill View

```
mysql> call show_room_bill('b102', 10);
+-----+
| user_id | first_name | last_name | phone_no |
+-----+
| u102    | William   | Jamie    | 2360291587 |
+-----+
1 row in set (0.00 sec)

+-----+
| booking_id | room_id | user_id | check_in | check_out |
+-----+
| b102       | r102    | u102    | 2022-11-15 | 2022-11-16 |
+-----+
1 row in set (0.00 sec)

+-----+
| Tax Percent |
+-----+
|          10 |
+-----+
1 row in set (0.01 sec)

+-----+
| Grand Total |
+-----+
|          550 |
+-----+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)
```

USER First Name, User_ID - Indices

We load the values from the User.csv file into the USER table and notice that it now contains 8959 entries along with the old ones.

```
mysql> select count(*) from USER;
+-----+
| count(*) |
+-----+
|      8959 |
+-----+
1 row in set (0.01 sec)
```

We apply indexing on the user_id and the first name as well. This is because usually, a receptionist checks the booking database on a first-name basis. To test the speed of the indexing, we create a duplicate INDEX_USER table and copy the values of USER table into it.

```
-- Adding indices to speed the searching
create table INDEXED_USER like USER;
insert INDEXED_USER select * from USER;
```

```
alter table INDEXED_USER add index(user_id);
alter table INDEXED_USER add index(first_name)
```

After adding the index, we see that executing a search query on the first name string has been sped up by 5 times.

```
mysql> show profiles;
```

Query_ID	Duration	Query
1	0.00533500	select * from USER where first_name LIKE 'B_a%'
2	0.00145300	select * from INDEXED_USER where first_name LIKE 'B_a%'

EXPENSES Amount - Indices

It is often convenient to filter the expenses based on the amount spent for that category. So, it speed up searching we add index to the amount in EXPENSES.

```
alter table EXPENSES add index(amount)
```

It speeds the following query as given in the table below:

Query_ID	Duration	Query
23	0.00861175	select * from expenses where amount < 150 and amount > 110
26	0.00512400	select * from expenses where amount < 150 and amount > 110

Extra Queries

Tardiness Alert - Trigger

When a staff logs his entry and exit times in the STAFF_LOG table, we can immediately find if he has arrived late (tardy) or left early. Based on this, we can keep track for each staff member how many times each occurrence has taken place. This is done using a trigger that is called after every insert into the STAFF_LOG table.

```
create trigger check_log after insert on STAFF_LOG
for each row
begin
    if (select shift_start from STAFF where STAFF.staff_id = NEW.staff_id)
    < NEW.entry_time then
        update STAFF set STAFF.late_entry_count = STAFF.late_entry_count + 1
    where STAFF.staff_id = NEW.staff_id;
    end if;
```

```
if (select shift_end from STAFF where STAFF.staff_id = NEW.staff_id) >
NEW.exit_time then
    update STAFF set STAFF.early_exit_count = STAFF.early_exit_count + 1
where STAFF.staff_id = NEW.staff_id;
end if;
End;|
```

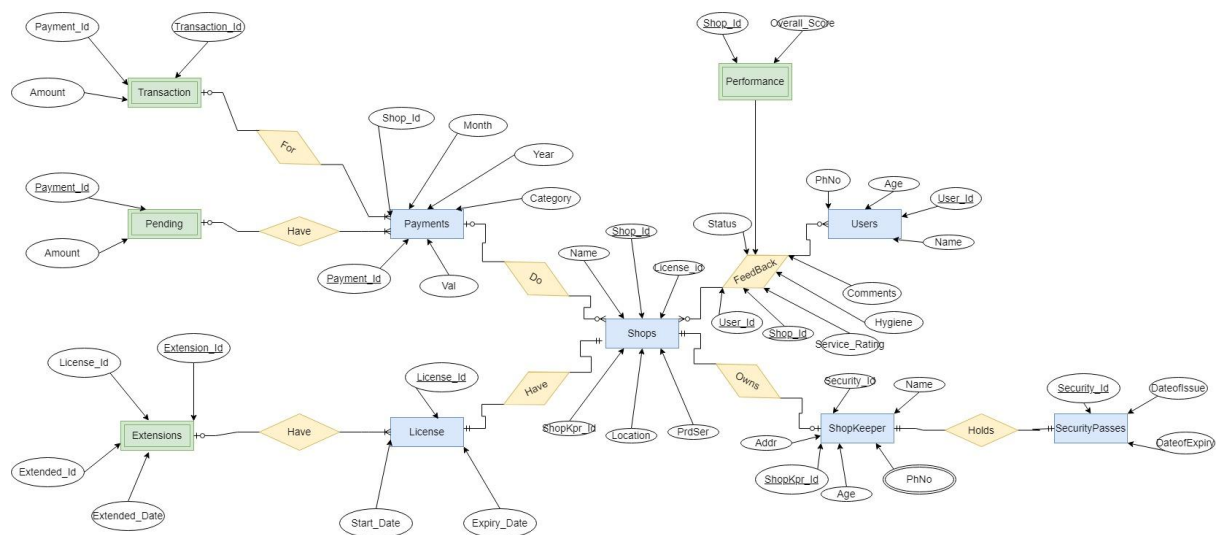
Room Wise Expenditure - Function

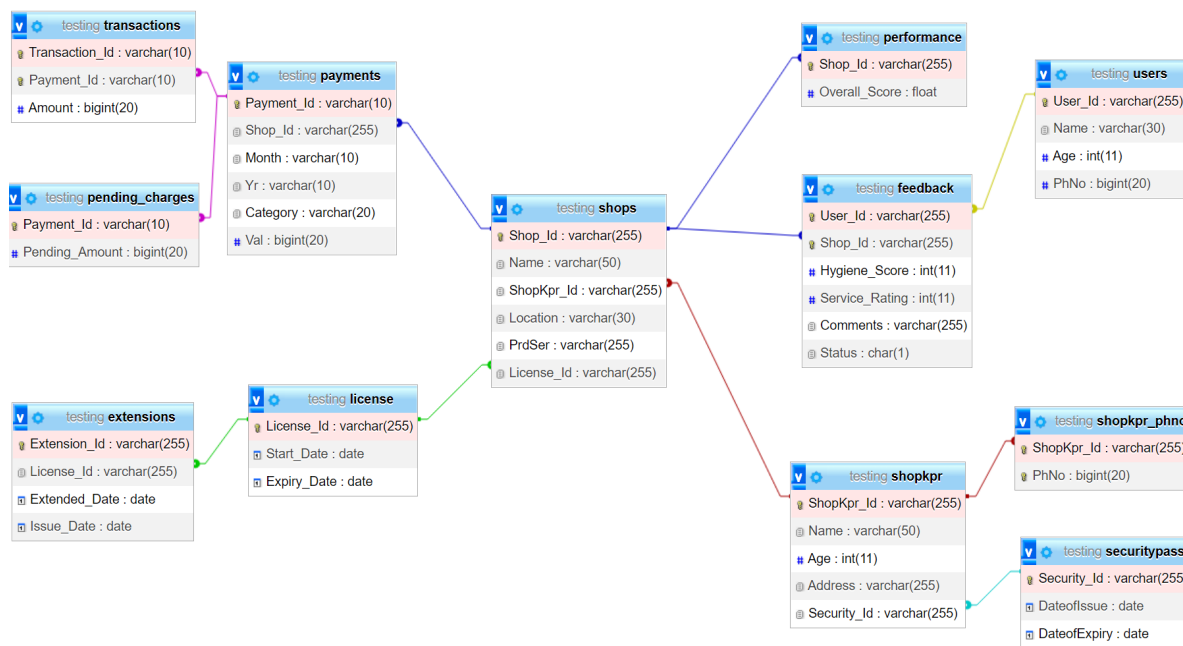
We can create a function to find the total expenditure incurred per room by summing the individual category-wise expenditure of 'Electricity', 'Water', 'Cleaning' present in the EXPENSES table.

```
create function calc_expenditure(room_id varchar(5)) returns bigint
deterministic
begin
    declare expd bigint;
    set expd = (select sum(EXPENSES.amount) from EXPENSES where
EXPENSES.room_id = room_id);
    return expd;
end;|
```

Market Shop Services

E-R Diagram:





● List Of Tables:

- **Shops** : Used to keep track of all shops in the market.
- **ShopKprs**: Used to keep track of all Shopkeepers in the market.
- **Users**: Used to keep records of regular users to various shops in the market.
- **FeedBack**: Used to keep records of all feedbacks/ratings given by users to various shops.
- **Performance**: Keeps record of overall performance metric of each shop, based on the Feedbacks.
- **SecurityPass**: Keeps track of security passes currently held by Shopkeepers.
- **ShopKpr_Phno**: Normalised table to accommodate the multi values attribute PhNo of shopKeepers.
- **Payments**: Keeps record of all the payments to be done by each shop under the categories {'Rent','Electricity'} and for each month.
- **Transactions**: Keeps record of all transactions made to administration for various Payments.
- **Pending_Charges**: Keeps record of how much of each payment is pending.

- **Licence:** Keeps record of licence held by each shop along with their expiry date.
- **Extensions:** Keeps record of various extensions given to all the licences.

Data

Table	No. of Rows
Extensions	18
FeedBack	821
License	157
Payments	20
Pending_Charges	20
Performances	4
Security_Pass	107
ShopKpr	107
ShopKpr_PhNo	156
Shops	157
Transactions	105
Users	1006

Queries:

Current Shop Details of different areas of the campus

- We have written the **procedure** 'Area_Details' which takes in the area name as input and projects the shop details whose location is same as that of the input's. Code:

```

delimiter |
create procedure Area_Details(IN Lcn varchar(30) )
begin
    select
Shops.Shop_Id,Shops.Name,Shops.ShopKpr_Id,ShopKpr.Name,Shops.
PrdSer from ( Shops inner join ShopKpr on ShopKpr.ShopKpr_Id
= Shops.ShopKpr_Id) where (Shops.Location = Lcn);
end;
|

```

- We applied inner join on Shop and ShopKeeper to get the details of the shopkeeper as well, and used a where condition of location being equal to input to get details of all shops in given area.

Name	ShopKpr_Id	Location	PrdSer	License_Id
Nescafe	001	Acad	Cafe	001
Nescafe	001	Acad	Cafe	002
Cycle	003	Hostel	Cycle	003
FoodCourt	002	Library	Food	004
Stationary	004	Quarters	Stationary	005
Icecream	005	Hostel	Icecream	006
Saloon	006	Hostel	Hairstyle	007

```
call Area_Details('Hostel');
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: Filter

Extra options

Shop_Id	Name	ShopKpr_Id	Name	PrdSer
003	Cycle	003	Praveen	Cycle
006	Icecream	005	Praveen	Icecream
007	Saloon	006	Praveen	Hairstyle

- **Additional Query:** We created another procedure to give us shop details if input is Shop_Id, using similar logic as above. Code:

```
create procedure Shop_Details(IN Id varchar(30))
begin
    select
Shops.Shop_Id,Shops.Name,Shops.ShopKpr_Id,ShopKpr.Name,Shops.
PrdSer from ( Shops inner join ShopKpr on ShopKpr.ShopKpr_Id
= Shops.ShopKpr_Id) where(Shops.Shop_Id = Id);
end;
|
```

```
call Shop_Details('001');
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25

Extra options

Shop_Id	Name	ShopKpr_Id	Name	PrdSer
001	Nescafe	001	Praveen	Cafe

Details of shop keepers and their security pass validity

- We have written another procedure 'ShpKpr_Details()' that takes ShpkPr_Id as input, and gives us the details of Shopkeepers along with the validity of his/her security pass .Code:

```

create procedure ShpKpr_Details(IN Id varchar(255))
begin

    select ShopKpr.ShopKpr_Id,
ShopKpr.Name,ShopKpr.Age,ShopKpr.Address,ShopKpr.Security_Id,
SecurityPass.DateofExpiry as PassExpiryDate from
(SecurityPass inner join ShopKpr on ShopKpr.Security_Id
=SecurityPass.Security_Id) where(ShopKpr.ShopKpr_Id= Id);

end;
|

```

- We again take help of inner join, between securitypass and shpKpr tables.

ShopKpr_Id	Name	Age	Address	Security_Id
001	Praveen	34	IIT Patna	001
002	Praveen	34	IIT Patna	002
003	Praveen	34	IIT Patna	003
004	Praveen	34	IIT Patna	004
005	Praveen	34	IIT Patna	005
006	Praveen	34	IIT Patna	006
007	Praveen	34	IIT Patna	007

-

```
call ShpKpr_Details('001');
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

ShopKpr_Id	Name	Age	Address	Security_Id	PassExpiryDate
001	Praveen	34	IIT Patna	001	2023-09-20

- **Additional Procedures:** We have implemented two additional procedures:

-> **Expire_Pass()**: Gives us list of Shopkeepers whose pass has expired or is going to within the next 5 days. It loops through the rows of SecurityPass with the help of Cursor. Code:

```
create procedure Expire_Pass()
begin
    declare done int default 0;
    declare S_ID varchar(255) ;
    declare S_Iss, S_Exp date;
    declare C_Date date;

    declare Pcursor CURSOR
    for select * from SecurityPass ;
    declare continue handler for not
found set done = 1;
    open Pcursor;
    set C_Date = CURDATE();

    label: LOOP
        fetch Pcursor into
        S_ID,S_Iss,S_Exp;
```

```
        if done = 1 then
            leave label;
        else

            if ((SELECT DATEDIFF(
S_Exp,C_Date)) <=5) then
                select S_ID,
ShopKpr.ShopKpr_Id,ShopKpr.Name from
(SecurityPass inner join ShopKpr on
ShopKpr.Security_Id =
SecurityPass.Security_Id) where
SecurityPass.Security_Id = S_Id;
            end if;

        end if;
    end loop;
    close Pcursor;

end;
|
```

▼	Security_Id	DateofIssue	DateofExpiry
Delete	001	2022-09-20	2023-09-20
Delete	002	2022-09-20	2023-09-20
Delete	003	2022-09-20	2023-09-20
Delete	004	2022-09-20	2023-09-20
Delete	005	2022-09-20	2023-09-20
Delete	006	2022-09-20	2023-09-20
Delete	007	2022-09-20	2023-09-20

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0007 seconds.)

[call](#) Expire_Pass();

> **Check_Pass(Id):** Determines whether the security pass, corresponding to shopkeeper with ID =Id, has expired or not.Code:

```
create procedure Check_Pass(IN Id varchar(255) )
begin
    declare C_Date date;

    declare MsgA, MsgB, MsgC varchar(30);
    declare diff int ;
    set C_Date = CURDATE();

    set diff = DATEDIFF( C_Date, (SELECT
SecurityPass.DateofExpiry from ( SecurityPass inner join
ShopKpr on ShopKpr.Security_Id =SecurityPass.Security_Id)
where ShopKpr.ShopKpr_Id = Id));
```

```

set MsgA = "Pass Expired";
set MsgB = "Valid Pass";

if(diff>0) then
    select MsgA;
else
    select MsgB;
end if;
end;
|

```

✓ Showing rows 0 - 0 (1 total, Query took 0.0006 seconds.)

```
call Check_Pass('001');
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: 25 Filter rows

Extra options

MsgB

Valid Pass

Reminders for expiring license agreement period

- Created a procedure 'Expire_License()' which loops through the License table, with help of cursor and checks if the expiry date of a licence is within 5 days of current date, or has been expired, if so, then it selects it.
- Have used CURDATE() to find current date.

- Code:

```
create procedure Expire_License() /** procedure to implement  
different operations based on input by if else statements**/  
begin  
    declare done int default 0; /* declare loop end  
indicator*/  
    declare L_ID varchar(255) ;/* variables to fetch the  
entries into*/  
    declare L_Sd,L_Exp date;  
    declare C_Date date;  
  
    declare P_cursor cursor for select * from License; /*  
declare cursor*/  
    declare continue handler for not found set done = 1;  
/* sets done =1 if no entry if fetched*/  
    open P_cursor;  
    set C_Date = CURDATE();  
    label: LOOP /* while loop*/  
    fetch P_cursor into L_ID,L_Sd, L_Exp;  
    if done = 1 then  
        leave label;  
    else  
  
        if ((SELECT DATEDIFF( L_Exp,C_Date)) <=5) then  
            select L_ID,  
Shops.Shop_Id,Shops.Name,ShopKpr.ShopKpr_Id,ShopKpr.Name from  
(ShopKpr inner join (License inner join Shops on  
Shops.License_Id = License.License_Id) on ShopKpr.ShopKpr_Id  
= Shops.ShopKpr_Id) where License.License_Id = L_Id;  
            end if;  
  
        end if;  
    end loop;  
    close P_cursor;
```

```
end; |
```

	▼ License_Id	Start_Date	Expiry_Date
 Delete	001	2022-09-20	2023-09-20
 Delete	002	2022-09-20	2022-11-20
 Delete	003	2022-09-20	2023-10-30
 Delete	004	2022-09-20	2023-09-30
 Delete	005	2022-09-20	2023-09-20
 Delete	006	2022-09-20	2023-10-20
 Delete	007	2022-09-20	2023-10-20

✓ Showing rows 0 - 0 (1 total, Query took 0.0012 seconds)

```
call Expire_License();
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: 25 ▼ Filter

Extra options

L_ID	Shop_Id	Name	ShopKpr_Id	Name
002	002	Nescafe	001	Praveen

Additional Trigger: Created an trigger 'ins_extension' that updates the licence table whenever an entry is made into extension table. It takes in the new expiry date from input and updates the licence table. This way there is no inconsistency.

Code:

```
create trigger ins_extension before insert on Extensions
for each row
begin
    update License set License.Expiry_Date =
new.Extended_Date where License.License_ID = new.License_ID;
End;
```

✓ 1 row inserted. (Query took 0.0003 seconds.)

```
insert into extensions VALUES ('008','001','2024-10-09','2022-11-09');
```

	License_Id	Start_Date	Expiry_Date
 Delete	001	2022-09-20	2024-10-09

Pending charges from each shop

- Created a procedure 'Pending_Summary' that gives the details of each shop that has some pending amount / entry in Pending_Charges table. This is implemented with the help of group by (Shop_Id) to get the total pending amount for each shop along with inner join to get details of teh Shop_Id.
- Code:

```
create procedure Pending_Summary()
```

```

begin
    call Update_Pendings();
    select Payments.Shop_Id,
sum(Pending_Charges.Pending_Amount) from (Pending_Charges
inner join Payments on Payments.Payment_Id =
Pending_Charges.Payment_Id) group by(Payments.Shop_Id);
end;
|

```

Payment_Id	Shop_Id	Month	Yr	Category	Val
001	001	Jan	2020	Rent	2000
002	003	Jan	2020	Rent	2000
003	001	Jan	2020	Rent	3000
004	002	Mar	2020	Rent	4000
005	003	May	2020	Rent	5000

Transaction_Id	Payment_Id	Amount
001	001	150
002	001	150
005	001	150
003	002	150
004	003	150

✓ Showing rows 0 - 2 (3 total, Query took 0.0068 seconds.)

`call Pending_Summary();`

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: Filter rows

Extra options

Shop_Id	sum(Pending_Charges.Pending_Amount)
001	4400
002	4000
003	6850

- **Additional Procedures and Triggers:**

->**Due_Amnt(Id)**: Created a function that returns that total pending amount for Shop with given Id. Implementation is similar to above.

->**Insert and Delete triggers**: Created triggers 'ins_trans' and 'del_trans' that update the Pending_Charges table whenever transactions are inserted or deleted. Thus, holding consistency.

Code:

```
delimiter |
create trigger ins_trans before insert on Transactions
for each row
begin

declare Amnt bigint ;
declare Pend bigint ;
```

```

    select Val into Amnt from Payments where Payments.Payment_Id
= new.Payment_Id;
    if exists (select Pending_Charges.Payment_Id from
Pending_Charges where Pending_Charges.Payment_Id =
new.Payment_Id) then
        begin
            select Pending_Charges.Pending_Amount into Pend from
Pending_Charges where Pending_Charges.Payment_Id =
new.Payment_Id;
            if(Pend>new.Amount) then
                update Pending_Charges set
Pending_Charges.Pending_Amount =
Pending_Charges.Pending_Amount - new.Amount where
Pending_Charges.Payment_Id = new.Payment_Id;
            elseif(Pend = new.Amount) then
                delete from Pending_Charges where
Pending_Charges.Payment_Id = new.Payment_Id;
            end if;
        end;

    elseif (Amnt > new.Amount) then

        insert into Pending_Charges
values(new.Payment_Id,(Amnt-new.Amount));

end if;
end;
|

create trigger del_trans before delete on Transactions
for each row
begin

    declare Pend bigint;
    if exists (select Pending_Charges.Payment_Id from
Pending_Charges where Pending_Charges.Payment_Id =
old.Payment_Id) then

```

```

        update Pending_Charges set
Pending_Charges.Pending_Amount =
Pending_Charges.Pending_Amount + old.Amount where
Pending_Charges.Payment_Id = old.Payment_Id;

    else
        insert into Pending_Charges
values(old.Payment_Id,old.Amount);

end if;
end;
|

```

Payment_Id	Pending_Amount
001	1550
002	1850
003	2850
004	4000
005	5000

->**Update_Pendings**: Created another procedure that updates pending charges for all the payments which did not have a single transactions for them.

Code:

```

create procedure Update_Pendings()
begin
    declare done int default 0;
    declare P_Id varchar(10);
    declare Sm bigint default 0;
    declare P_amnt bigint default 0;

```

```

    declare P_cursor cursor for select Payment_Id from
Payments;
    declare continue handler for not found set done =1;
    open P_cursor;
    Label: LOOP
        fetch P_cursor into P_Id;

        if done =1 then
            leave Label;
        else

            if (not exists (select Payment_Id from
Pending_Charges where Payment_ID = P_Id) ) then
                select Val into P_amnt from Payments where
Payment_Id = P_Id;
                insert into pending_charges values
(P_Id,P_amnt-Sm);
            end if;

        end if;
    end LOOP;
    close P_cursor;
end;

```

->Part of Pending_Summary.

Summary of performances of each shop

- Created procedure 'Performance_Summary' which gives all the feedbacks made for a Shop with given Id. MYSQL query involves inner join between feedback and Shops table. Code:

```

create procedure Performance_Summary(IN S_Id varchar(255))

begin

```

```

select
Shops.Shop_Id,Shops.Name,FeedBack.Hygiene_Score,FeedBack.Serv
ice_Rating,FeedBack.Comments from ( Shops inner join FeedBack
on Shops.Shop_Id = FeedBack.Shop_Id) where
((FeedBack.Shop_Id = S_Id)and(FeedBack.Status='A'));

end;
|

```

User_Id	Shop_Id	Hygiene_Score	Service_Rating	Comments	Status
001	001	3	5	GOOD	P
001	003	3	5	Good	P
001	005	3	5	Good	P
002	001	3	5	Ok	A
002	004	3	5	Bad	A
003	002	3	5	Bad	P
004	003	3	5	Normal	A
006	002	3	5	Normal	A

```
call performance_summary('002');
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: 25 Filter rows:

[Extra options](#)

Shop_Id	Name	Hygiene_Score	Service_Rating	Comments
002	Nescafe	3	5	Normal

- **Additional triggers/procedures:**

-> **Insert, update and delete triggers:** Whenever a feedback that has been accepted (Status = 'A') is inserted or if it has been updated or status has been revoked, the corresponding change is made to the performance table's overall_score column. Code:

```
create trigger ins_score before insert on FeedBack
for each row
begin
if ( (new.Status = 'A') and (exists ( select
Performance.Shop_Id from Performance where
Performance.Shop_Id = new.Shop_Id) ) ) then
    update Performance set Performance.Overall_Score =
Performance.Overall_Score + ( (new.Hygiene_Score
+new.Service_Rating)/2) where Performance.Shop_Id =
new.Shop_Id;
elseif ( (new.Status = 'A') and ( not exists ( select
Performance.Shop_Id from Performance where
Performance.Shop_Id = new.Shop_Id) ) ) then
    insert into Performance values
(new.Shop_Id, ((new.Hygiene_Score +new.Service_Rating)/2));
end if;
end;
```



```

|

create trigger upd_score before update on FeedBack
for each row
begin
if ((old.Status='A')and(new.Status='P')) then
    update Performance set Performance.Overall_Score =
Performance.Overall_Score - ((old.Hygiene_Score
+old.Service_Rating)/2) where ( (Performance.Shop_Id =
new.Shop_Id) );
elseif ((old.Status='A')and(new.Status='A')) then
    update Performance set Performance.Overall_Score =
Performance.Overall_Score + ((new.Hygiene_Score
+new.Service_Rating)/2) -((old.Hygiene_Score
+old.Service_Rating)/2) where ((Performance.Shop_Id =
new.Shop_Id));
elseif ((old.Status='P')and(new.Status='A')) then
    update Performance set Performance.Overall_Score =
Performance.Overall_Score + ((new.Hygiene_Score
+new.Service_Rating)/2) where ( (Performance.Shop_Id =
new.Shop_Id) );

```

```

end if;
end;
|

```

```

create trigger del_score after delete on FeedBack
for each row
begin
if ( (old.Status='A') ) then
    update Performance set Performance.Overall_Score =
Performance.Overall_Score - ((old.Hygiene_Score
+old.Service_Rating)/2) where      ( (Performance.Shop_Id =
old.Shop_Id) );

```

```

end if;
end;
|

```

```

create procedure del_comments()
begin
delete from FeedBack where status = 'P';
end;
|

```

	Shop_Id	Overall_Score
Delete	001	4
Delete	002	4
Delete	003	4
Delete	004	4

->**Get_Performance()**: A simple procedure to check if a shop has performance score, if it does then select it, else selects error message.Code:

```

create procedure Get_Performance(IN S_Id varchar(255))
begin
declare score float;
declare Err varchar(20);
set Err = "ID not present";
if exists( select Shop_Id from Performance where Shop_Id =
S_Id) then
select Shops.Shop_Id, Shops.Name,
Performance.Overall_Score from ( Performance inner join Shops
on Shops.Shop_Id = Performance.Shop_Id) where Shops.Shop_Id =
S_Id;
else
select Err ;
end if;

end;
|

```

```
call Get_Performance('001');
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows:

Extra options

Shop_Id	Name	Overall_Score
001	Nescafe	4

->**Ranklist()**: Sort the shop based on their performance score, by using order by keyword. Code:

```
create procedure RankList()
begin
    select Shops.Shop_Id, Shops.Name, Performance.Overall_Score
    from ( Performance inner join Shops on Shops.Shop_Id =
    Performance.Shop_Id) order by Performance.Overall_Score ;
end
|
```

```
call Ranklist();
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: 25

Extra options

Shop_Id	Name	Overall_Score
001	Nescafe	4
002	Nescafe	4
003	Cycle	4
004	FoodCourt	4

Not visible as all have same score

- **Additional Procedure (User_Comments):** Lists down all the comments/feedback given by a particular user_Id. Code:

```
delimiter |
create procedure User_Comments(IN Id varchar(255))
begin
    select Users.User_Id,FeedBack.Shop_Id
    ,Shops.Name,FeedBack.Comments from (Shops inner join ( Users
inner join FeedBack on Users.User_Id = FeedBack.User_Id) on
Shops.Shop_Id = FeedBack.Shop_Id) where Users.User_Id = Id;
end;
|
```

```
call User_Comments('001');
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: 25 ▼

Extra options

User_Id	Shop_Id	Name	Comments
001	001	Nescafe	GOOD
001	003	Cycle	Good
001	005	Stationary	Good

INDEXING

-> Added an index index_1(User_Id, Shop_Id) on table FeedBack.

```
CREATE index index_1 on feedback(User_Id,Shop_Id);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0021 seconds.)

```
call User_Comments('u108');
```